SRS — Biblioteca Inteligente

Nov 4, 2026 · @Mirllan

Información del documento

Este SRS define Biblioteca Inteligente, un catálogo web con votos en tiempo real y recomendaciones por IA, construido sobre la aplicación Django que ya funciona.

Campo	Valor
Curso	Electiva V
Entrega	13. Entrega 1: definición de la solución y la arquitectura
Versión	1.0
Integrantes	David Humberto Mahecha Cardozo y Edilson Mirllan Suarez Gutiérrez
Repositorio	github.com/David78-beto/Biblioteca-Inteligente
Video	youtube.com/watch?v=1tX8ytETkqM

1. Introducción

El documento sirve al equipo de desarrollo y al docente como acuerdo de qué se construye y con qué arquitectura. Su estructura se inspira en la norma ISO/IEC/IEEE 29148, adaptada al alcance de un proyecto académico.

Término	Definición
SRS	Especificación de requisitos de software
RF / RNF	Requisito funcional / requisito no funcional
Embedding	Vector numérico que representa el significado de un texto
WebSocket	Canal bidireccional que permite al servidor enviar datos al navegador

Término	Definición
Contenedor	Paquete aislado con una aplicación y sus dependencias, ejecutado con Docker

2. Problema a resolver

Los lectores de una biblioteca pequeña no tienen una forma sencilla de saber qué libros hay ni cuál leer a continuación.

- **Quién lo tiene:** lectores y bibliotecarios de bibliotecas escolares o comunitarias sin un sistema de gestión propio.
- **Cómo se ve actualmente:** el catálogo suele vivir en listas o en hojas de cálculo. Encontrar un libro exige preguntar o revisar los estantes, y las recomendaciones dependen de la memoria del bibliotecario.
- **Por qué vale la pena resolverlo:** un catálogo consultable reduce el trabajo repetitivo del bibliotecario, y las recomendaciones acercan a cada lector a libros que quizá nunca habría encontrado.

Pregunta abierta: este planteamiento es un supuesto del equipo. Conviene validarlo con al menos una biblioteca real antes de la Entrega 2.

3. Justificación

El proyecto se justifica porque resuelve una necesidad concreta con herramientas de código abierto y permite aplicar cada tecnología del curso a un problema distinto.

- **Valor para el usuario:** une en un solo lugar la consulta del catálogo, la opinión de otros lectores y las recomendaciones.
- **Costo bajo:** todo el software elegido es gratuito, y el modelo de IA se ejecuta localmente, sin pagar por una API externa.
- **Riesgo controlado:** el catálogo en Django ya funciona, de modo que las entregas siguientes agregan capacidades sobre una base probada.
- **Valor académico:** el equipo practica una arquitectura de servicios con Django, Node.js, IA, Docker y una base de datos relacional.

4. Objetivos

Objetivo general: desarrollar una aplicación web contenerizada que permita consultar y gestionar el catálogo de una biblioteca, y que recomiende libros mediante inteligencia artificial.

Objetivos específicos:

1. Gestionar géneros y libros con operaciones de consulta, creación, edición y eliminación en Django.
2. Registrar el interés de los lectores mediante votos y mostrarlos en tiempo real con un servicio en Node.js.
3. Recomendar libros similares y permitir búsquedas en lenguaje natural con un servicio de IA.
4. Ejecutar todos los servicios con Docker Compose mediante un único comando.
5. Mantener la calidad del código con pruebas automatizadas e integración continua en GitHub.

5. Alcance

El proyecto cubre el catálogo, los votos, las notificaciones y las recomendaciones de una sola biblioteca; no cubre los préstamos físicos.

Incluido:

- Catálogo de géneros y libros, con administración.
- Registro e inicio de sesión de usuarios, con un voto por libro y usuario.
- Votos y notificaciones en tiempo real.
- Recomendaciones por similitud y búsqueda en lenguaje natural.
- Despliegue local con Docker Compose.

Excluido:

- Préstamos, devoluciones y multas.
- Pagos y compra de libros.
- Lectura de libros digitales.
- Aplicación móvil nativa y despliegue en la nube.

Supuestos: el catálogo se escribe en español y cabe en un solo servidor.

Estado actual: la aplicación Django ya tiene el catálogo, los votos anónimos, la administración y las pruebas automatizadas. Los servicios de Node.js e IA, los usuarios y PostgreSQL en uso real están definidos aquí, pero aún no se han construido.

6. Tecnologías

Cada tecnología tiene una responsabilidad única, y solo Django escribe en la base de datos del catálogo.

Tecnología	Responsabilidad	Motivo de la elección
Django 6.1 (Python)	Interfaz web, reglas de negocio, administración y API REST	Ya es la base del proyecto; incluye ORM, administración y autenticación
Node.js con Socket.IO	Enviar votos y notificaciones en tiempo real al navegador	Maneja bien muchas conexiones abiertas y simultáneas
FastAPI con sentence-transformers	Servicio de IA: generar embeddings, recomendar y buscar	Modelo local y gratuito, con soporte para español
PostgreSQL	Base de datos de producción	Relacional, robusta y disponible como imagen de Docker
SQLite	Base de datos de desarrollo	Sin instalación; es la que usa hoy el proyecto
Docker y Docker Compose	Empaquetar y levantar todos los servicios	Mismo entorno en el equipo de cada integrante
Nginx	Punto de entrada único hacia Django y Node.js	Dirige el tráfico web y el de WebSocket
Git y GitHub Actions	Control de versiones e integración continua	Ejecuta las pruebas en cada envío de código

7. Requisitos funcionales

Ocho de los diecisiete requisitos ya están implementados en Django; los otros nueve se planifican para las entregas siguientes.

ID	El sistema debe…	Actor	Prioridad	Estado
RF-01	Listar los géneros disponibles	Lector	Alta	Implementado
RF-02	Listar los libros de un género	Lector	Alta	Implementado
RF-03	Mostrar el detalle de un libro	Lector	Alta	Implementado
RF-04	Permitir agregar un libro	Bibliotecario	Alta	Implementado
RF-05	Permitir editar un libro	Bibliotecario	Alta	Implementado
RF-06	Eliminar un libro tras pedir confirmación	Bibliotecario	Alta	Implementado
RF-07	Registrar un voto en un libro	Lector	Media	Implementado
RF-08	Administrar géneros y libros desde un panel	Administrador	Media	Implementado
RF-09	Permitir el registro y el inicio de sesión	Lector	Alta	Planificado
RF-10	Limitar a un voto por usuario y libro	Lector	Alta	Planificado
RF-11	Restringir la gestión del catálogo al personal autorizado	Bibliotecario	Alta	Planificado
RF-12	Exponer el catálogo mediante una API REST	Servicio de IA	Alta	Planificado
RF-13	Actualizar los votos en pantalla sin recargar	Lector	Media	Planificado

ID	El sistema debe…	Actor	Prioridad	Estado
RF-14	Notificar en pantalla la llegada de libros nuevos	Lector	Baja	Planificado
RF-15	Recomendar libros similares en la página de detalle	Lector	Alta	Planificado
RF-16	Buscar libros con una frase en lenguaje natural	Lector	Media	Planificado
RF-17	Guardar una descripción de cada libro, que el servicio de IA usa para recomendar y buscar	Bibliotecario	Alta	Planificado

8. Requisitos no funcionales

Los requisitos siguientes tienen una condición que se puede comprobar; las metas de tiempo son objetivos del equipo y aún no se han medido.

ID	Atributo	Requisito	Cómo se verifica
RNF-01	Rendimiento	Las páginas del catálogo responden en menos de 2 s en local	Medición con las herramientas del navegador
RNF-02	Tiempo real	Un voto se ve en otro navegador en menos de 1 s	Prueba manual con dos navegadores
RNF-03	Disponibilidad	Si el servicio de IA falla, el catálogo sigue funcionando	Prueba con el contenedor de IA detenido
RNF-04	Seguridad	Las claves se leen de variables de entorno y nunca se versionan	Revisión del repositorio
RNF-05	Seguridad	Toda acción que cambia datos usa POST con protección CSRF	Pruebas automatizadas
RNF-06	Portabilidad	El sistema se levanta con `docker compose up`	Ejecución en un equipo limpio

ID	Atributo	Requisito	Cómo se verifica
RNF-07	Mantenibilidad	Las pruebas se ejecutan en cada envío a GitHub	Registro de GitHub Actions
RNF-08	Usabilidad	La interfaz está en español y se adapta a pantallas de celular	Revisión a 375 px de ancho

9. Diagramas UML

Los tres diagramas muestran la arquitectura objetivo; en este momento solo Django y su base de datos están construidos.

Casos de uso

Cinco casos de uso ya funcionan y cuatro quedan planificados

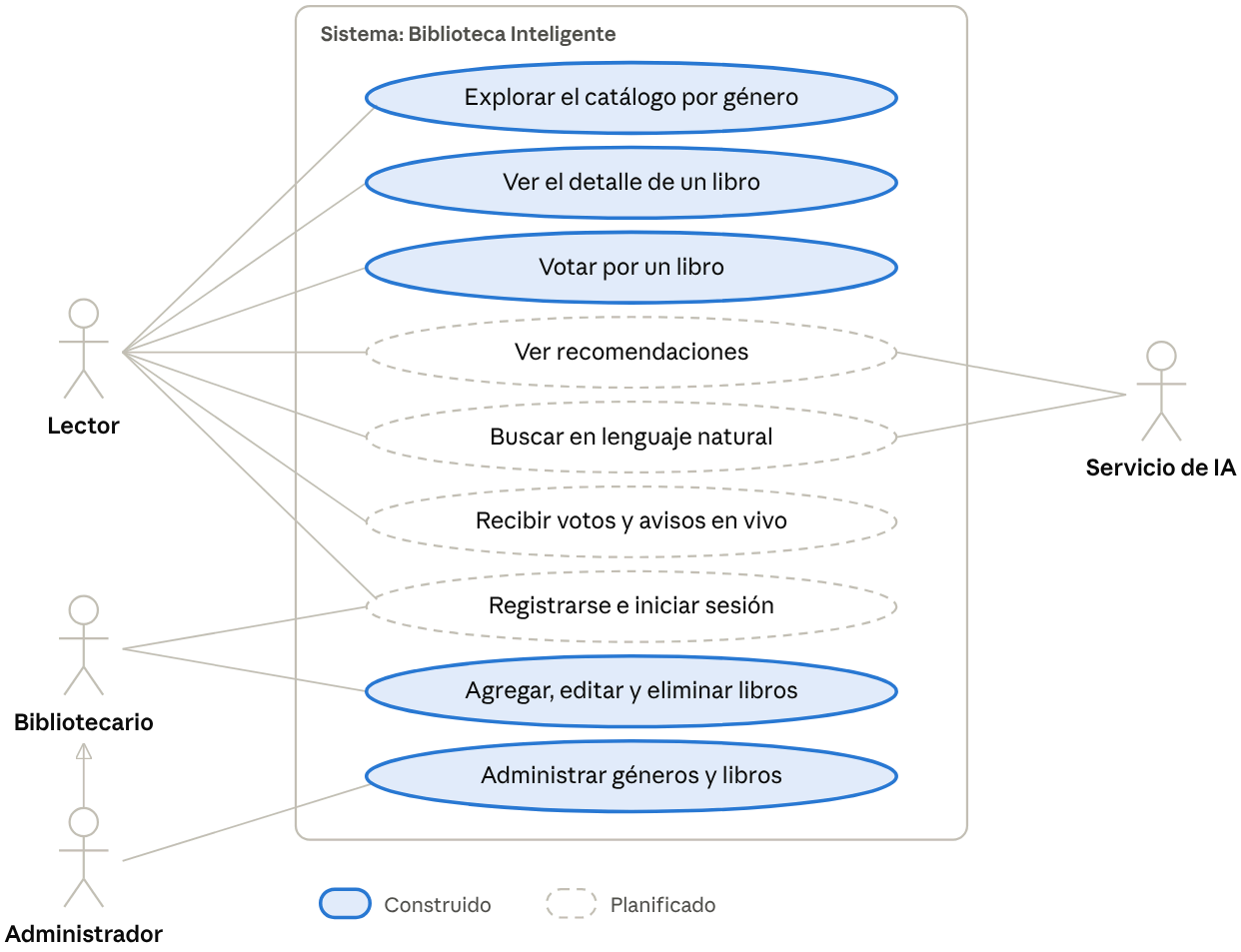


Diagrama de casos de uso · 9 casos, 4 actores

El Administrador hereda los casos del Bibliotecario. El Servicio de IA es un actor secundario: responde a las recomendaciones y a las búsquedas.

Componentes

Django es el núcleo; Node.js y la IA son servicios separados

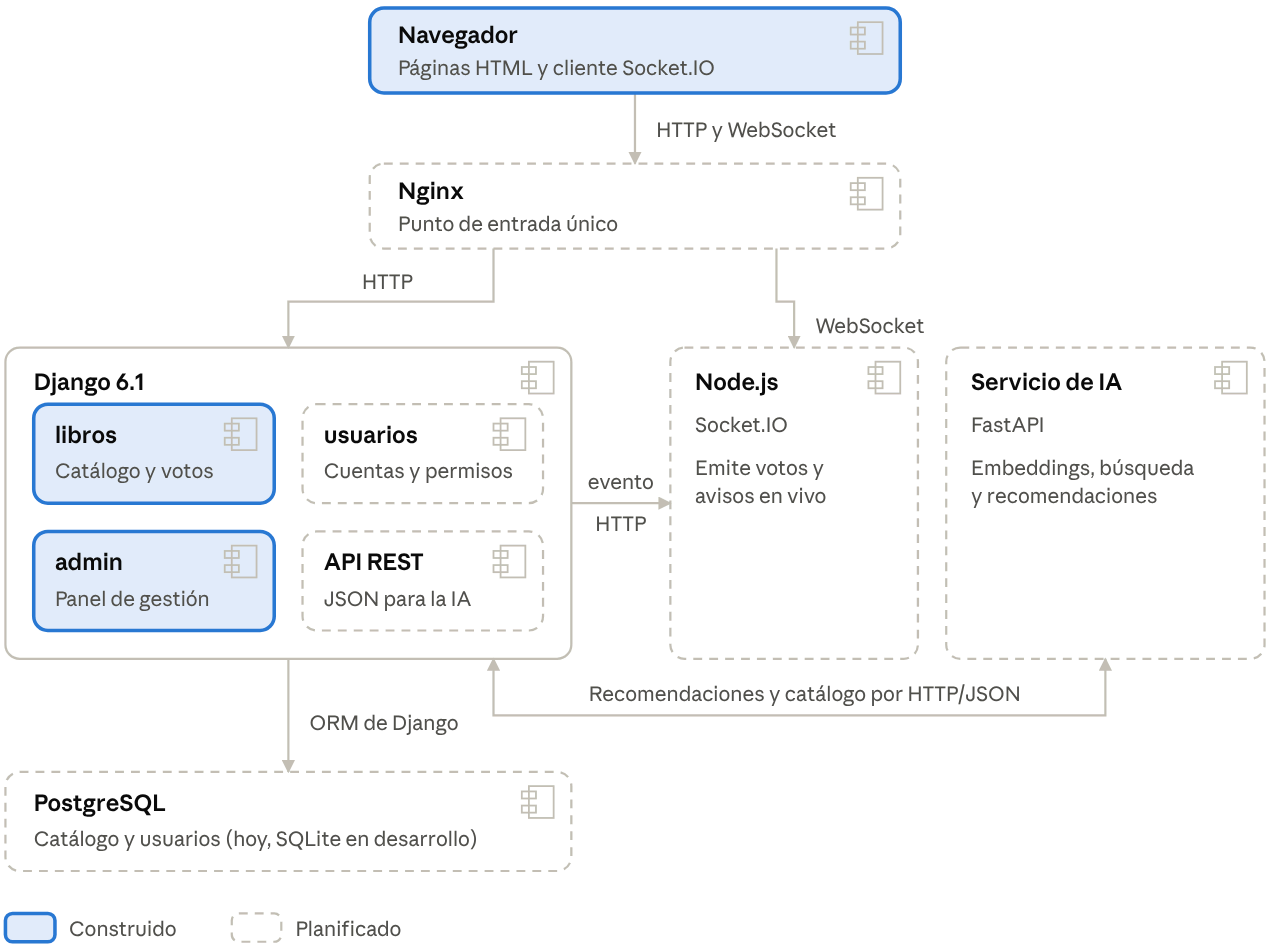


Diagrama de componentes · Django, Node.js, IA y base de datos

Al registrarse un voto, Django guarda el dato y avisa a Node.js, que lo reenvía a los navegadores conectados. Django pide las recomendaciones al servicio de IA, que obtiene el catálogo mediante la API REST y nunca escribe en la base de datos.

Despliegue

Solo Nginx recibe tráfico externo; los demás contenedores quedan en una red interna

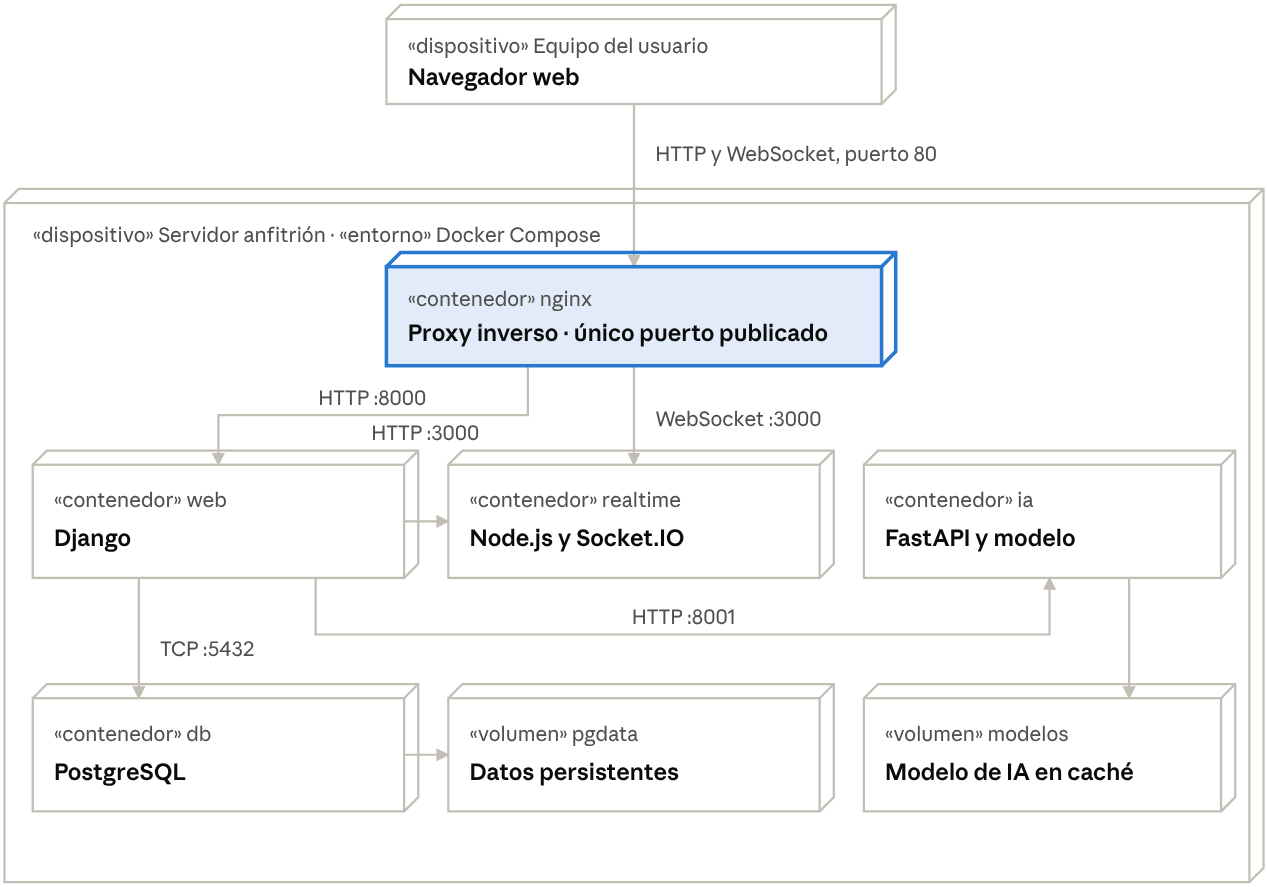


Diagrama de despliegue · 5 contenedores y 2 volúmenes

Docker Compose levanta los cinco contenedores en un solo servidor. Los volúmenes conservan los datos y el modelo de IA cuando un contenedor se reinicia.

10. Backlog inicial

El backlog tiene 13 historias ordenadas por prioridad: 4 ya están hechas y 9 quedan para las entregas siguientes.

ID	Historia de usuario	Criterio de aceptación	RF	Prioridad	Estado
HU-01	Como lector, quiero explorar los libros por género para encontrar qué leer	Al elegir un género veo solo sus libros; un género inexistente muestra error 404	RF-01, RF-02, RF-03	Alta	Hecho

ID	Historia de usuario	Criterio de aceptación	RF	Prioridad	Estado
HU-02	Como bibliotecario, quiero agregar, editar y eliminar libros para mantener el catálogo al día	Los cambios se guardan, se confirma cada acción y eliminar pide confirmación	RF-04, RF-05, RF-06	Alta	Hecho
HU-03	Como lector, quiero votar por un libro para mostrar que me gustó	El contador sube en 1 con cada voto enviado por POST	RF-07	Media	Hecho
HU-04	Como administrador, quiero un panel para gestionar géneros y libros	Puedo buscar y filtrar libros y crear géneros en `/admin/`	RF-08	Media	Hecho
HU-05	Como lector, quiero crear una cuenta para tener una identidad en el sistema	Puedo registrarme, entrar y salir; una contraseña incorrecta se rechaza	RF-09	Alta	Por hacer
HU-06	Como lector, quiero que mi voto cuente una sola vez para que el ranking sea justo	Un segundo voto del mismo usuario no cambia el contador	RF-10	Alta	Por hacer
HU-07	Como bibliotecario, quiero que solo el personal modifique el catálogo para proteger los datos	Un lector sin permiso recibe error 403 al intentar editar	RF-11	Alta	Por hacer
HU-08	Como servicio de IA, quiero leer el catálogo por API para generar los embeddings	`GET /api/libros/` devuelve los libros en JSON, con título, autor, género y descripción	RF-12	Alta	Por hacer
HU-13	Como bibliotecario, quiero registrar la descripción de cada libro para que la IA pueda recomendarlo y encontrarlo	El formulario guarda la descripción y el detalle la muestra; un libro sin descripción sigue siendo válido	RF-17	Alta	Por hacer
HU-09	Como lector, quiero recibir recomendaciones similares para descubrir más libros	El detalle muestra hasta 5 libros, sin incluir el actual; si la IA falla, la página carga sin ellos	RF-15	Alta	Por hacer
HU-10	Como lector, quiero ver los votos cambiar en vivo para saber qué interesa ahora	Un voto aparece en otro navegador sin recargar la página	RF-13	Media	Por hacer
HU-11	Como lector, quiero buscar con una frase para encontrar un libro sin saber su título	«Historia sobre un planeta desértico» incluye Dune entre los primeros resultados	RF-16	Media	Por hacer

ID	Historia de usuario	Criterio de aceptación	RF	Prioridad	Estado
HU-12	Como lector, quiero saber cuándo llega un libro nuevo para no perdérmelo	Con la página abierta aparece un aviso al agregarse un libro	RF-14	Baja	Por hacer